

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0508

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 30%

Dr.G.Ayyappan

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.	BL3 – Apply	5
2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.	BL3 – Apply	5
3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.	BL3 – Apply	5
4	Debug the given relational algebra expression that produces incorrect output for a natural join operation.	BL3 – Apply	5
5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.	BL3 – Apply	5
6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.	BL6 – Create	5
7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.	BL3 – Apply	5

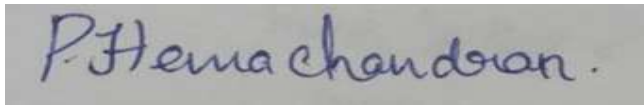
8	Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.	BL3 – Apply	5
9	Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes.	BL6 – Create	5
10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.	BL3 – Apply	5

Code of Conduct:

I P. HEMACHANDRAN (192525440)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

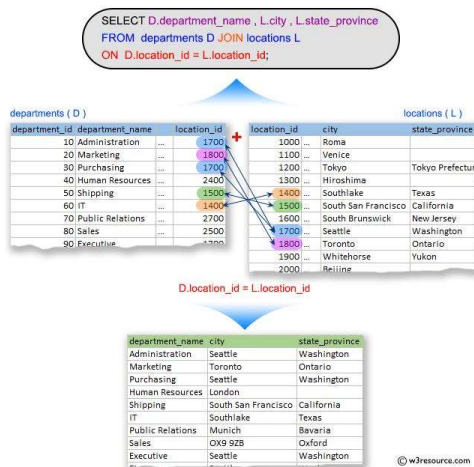


RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained

Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

1. Incorrect JOIN Syntax Returning Duplicate Results



Correct SQL

```
SQL

SELECT E.EmpID,
       E.EmpName,
       D.DeptName
FROM Employee E
INNER JOIN Department D
ON E.DeptID = D.DeptID;
```

Incorrect SQL

```
SQL

SELECT *
FROM Employee, Department;
```

This creates a **Cartesian Product**, where every employee is matched with every department, resulting in duplicate or unnecessary rows.

Theory

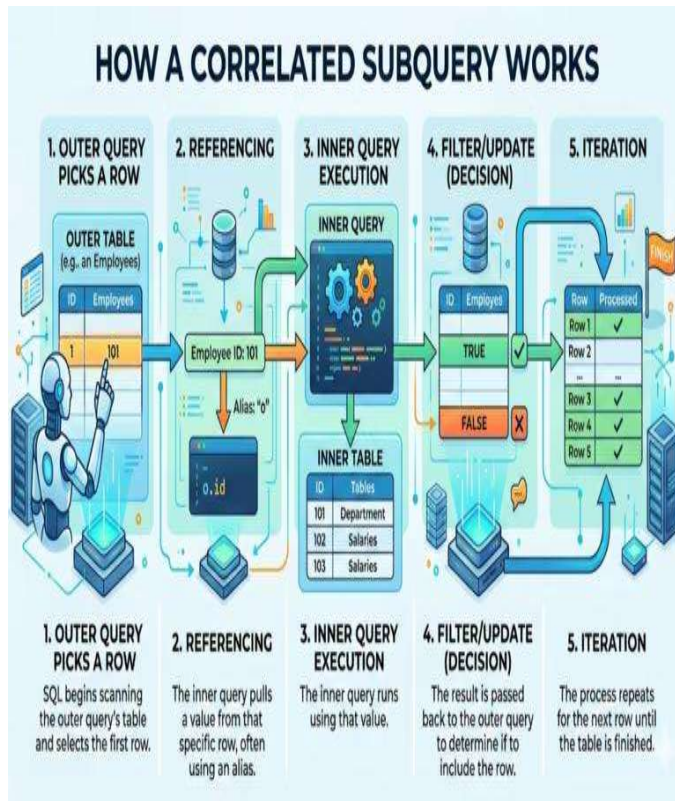
Duplicate results usually occur when tables are joined without a proper join condition or when an incorrect JOIN syntax is used.

Incorrect SQL

Explanation

- Use **INNER JOIN** with the correct join condition.
- Avoid comma-separated table lists unless performing a Cartesian Product intentionally.
- Ensure the join uses matching primary and foreign keys.

2. Sub-query Missing Correlation



Incorrect SQL

<> SQL

```
SELECT *
FROM Employee
WHERE Salary >
(
SELECT AVG(Salary)
FROM Employee
);
```

Correct SQL

<> SQL

```
SELECT *
FROM Employee E
WHERE Salary >
(
SELECT AVG(Salary)
FROM Employee
WHERE DeptID = E.DeptID
);
```

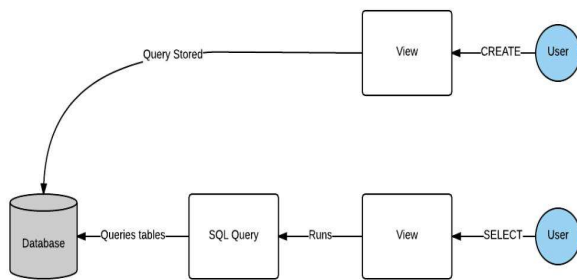
Theory

A correlated sub-query depends on the outer query. Without correlation, the sub-query computes a single value for the entire table, leading to incorrect results.

Explanation

The corrected query compares each employee's salary with the average salary of **their own department**, not the entire organization.

3. SQL View Causing Errors



Incorrect View

</> SQL

```
CREATE VIEW EmpView AS
SELECT *
FROM Employee
JOIN Department;
```

Correct View

</> SQL

```
CREATE VIEW EmpView AS
SELECT
  E.EmpID,
  E.EmpName,
  D.DeptName
FROM Employee E
JOIN Department D
ON E.DeptID = D.DeptID;
```

Theory

A view should avoid ambiguous column names and unnecessary complexity.

Explanation

- Avoid using ****SELECT **** in views.
- Specify required columns.
- Always include proper JOIN conditions.

4. Relational Algebra Expression for Natural Join

Students x Courses

Id	Name	Students.Course_id	Courses.Course_id	Course_name
1	Leslie	1	1	Math
1	Leslie	1	2	Art
2	John	2	1	Math
2	John	2	2	Art
3	Amy	1	1	Math
3	Amy	1	2	Art
4	Cindy	2	1	Math
4	Cindy	2	2	Art



$\sigma_{Students.course_id=Courses.id}(Students \times Courses)$

Id	Name	Course_id	Course_name
1	Leslie	1	Math
2	John	2	Art
3	Amy	1	Math
4	Cindy	2	Art

Incorrect Expression

Employee x Department

This performs a Cartesian Product instead of joining matching records.

Correct Expression

Employee ⋈ Department

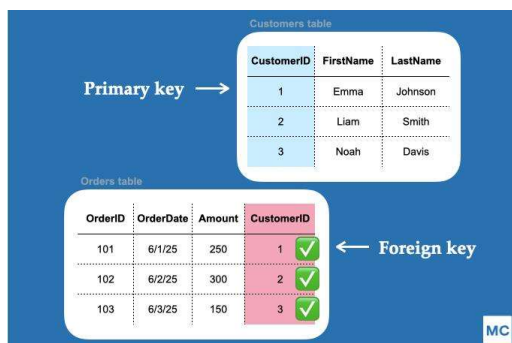
Explanation

Natural Join combines tuples having the same value in common attributes and eliminates duplicate columns.

Theory

A **Natural Join** ($\bowtie$) automatically joins two relations based on common attribute names.

5. Referential Integrity Violation



Theory

Referential integrity ensures that every foreign key value matches an existing primary key value.

Incorrect SQL

```
</> SQL
INSERT INTO Employee
VALUES
(101, 'Ravi', 20);
```

Suppose Department 20 does not exist.

Correct SQL

```
</> SQL
INSERT INTO Department
VALUES
(20, 'IT');

INSERT INTO Employee
VALUES
(101, 'Ravi', 20);
```

Explanation

- Insert the parent record first.
- Define a FOREIGN KEY constraint.
- Prevent orphan records.

6. Optimizing Nested Queries Using JOIN

Example:

Inefficient Query:

```
SELECT CustomerName
FROM Customers
WHERE CustomerID IN (
    SELECT CustomerID FROM Orders WHERE Amount > 500
);
```

Optimized Query:

```
SELECT Customers.CustomerName
FROM Customers
JOIN Orders ON Customers.CustomerID = Orders.CustomerID
WHERE Orders.Amount > 500;
```

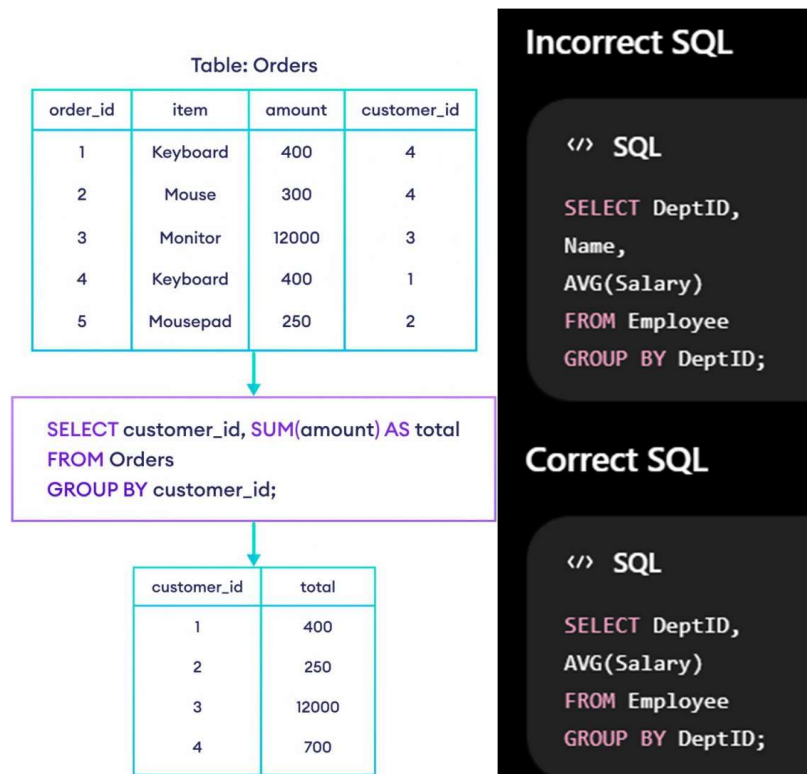
Theory

JOIN operations are generally faster than deeply nested sub-queries because database optimizers can generate efficient execution plans.

Benefits

- Faster execution.
- Better readability.
- Efficient index utilization.

7. Incorrect GROUP BY Aggregation



Theory

Every non-aggregated column in the SELECT clause must appear in the GROUP BY clause.

Explanation

Failing to group by all non-aggregated columns results in SQL errors or incorrect aggregation.

8. SQL UPDATE Statement Modifies All Rows

Table: Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

```
UPDATE Customers
SET first_name = 'Johnny'
WHERE customer_id = 1;
```

customer_id	first_name	last_name	age	country
1	Johnny	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Harry	Potter	31	USA

Incorrect SQL

```
</> SQL
```

```
UPDATE Employee
SET Salary=Salary*1.10;
```

Correct SQL

```
</> SQL
```

```
UPDATE Employee
SET Salary=Salary*1.10
WHERE DeptID=10;
```

Theory

An UPDATE statement without a WHERE clause modifies every record in the table.

Explanation

Always include a WHERE clause unless updating every row intentionally.

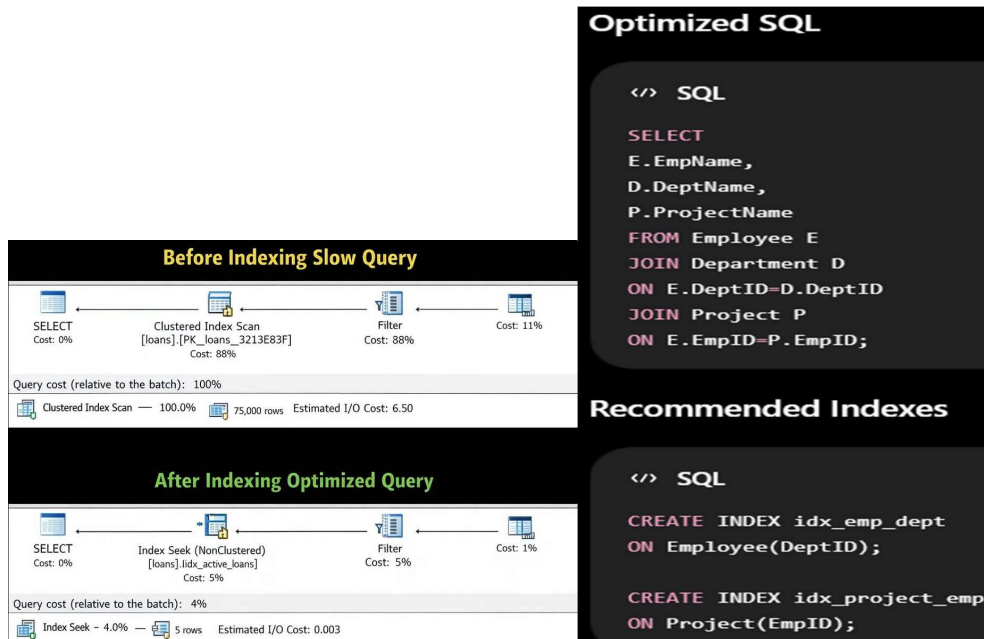
9. Optimizing Multi-table Query

Theory

Performance improves by replacing nested sub-queries with JOIN operations and creating indexes on frequently searched columns.

Benefits

- Faster searching.
- Reduced execution time.
- Efficient query optimization.



10. Relational Calculus Quantifier Error

Find sailors who've reserved all boats

$$\{S \mid S \in \text{Sailors} \wedge \forall B (B \in \text{Boats} \Rightarrow \exists R (R \in \text{Reserves} \wedge S.\text{sid} = R.\text{sid} \wedge B.\text{bid} = R.\text{bid}))\}$$

Find all sailors S such that for each tuple B either it is not a tuple in Boats or there is a tuple in Reserves showing that sailor S has reserved it.

$$\{S \mid S \in \text{Sailors} \wedge \forall B (\neg(B \in \text{Boats}) \vee \exists R (R \in \text{Reserves} \wedge S.\text{sid} = R.\text{sid} \wedge B.\text{bid} = R.\text{bid}))\}$$

Incorrect Expression

$$\forall x (Employee(x) \rightarrow x.Dept = 'IT')$$

This requires every employee to belong to the IT department, which is usually false.

Correct Expression

$$\exists x (Employee(x) \wedge x.Dept = 'IT')$$

Theory

Tuple Relational Calculus uses logical quantifiers:

- $\exists$ (There Exists) – At least one tuple satisfies the condition.
- $\forall$ (For All) – Every tuple must satisfy the condition.

Using the wrong quantifier can produce an empty result.

Explanation

The existential quantifier ($\exists$) correctly retrieves employees who work in the IT department without requiring all employees to satisfy the condition.